

10. ROS2 Custom Interface Messages

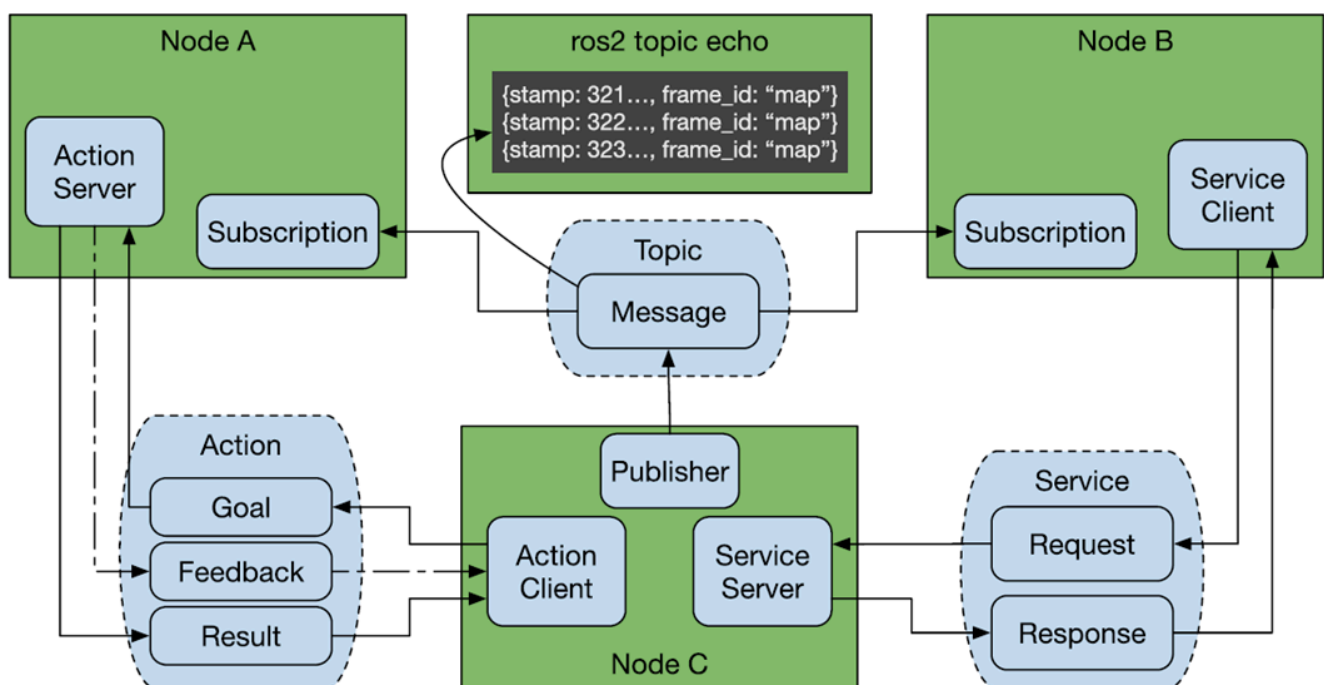
Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Communication Interface Overview

In the ROS system, whether it's topics, services, or actions, an important concept is used - the communication interface.

Communication is not one person talking to themselves, but two or more people exchanging ideas. What is being exchanged? To make it easy for everyone to understand, we can define a standard structure for the transmitted data. This is the communication interface.

Interfaces can reduce dependencies between programs, making it easy for us to use others' code and for others to use our code. This is the core goal of ROS - reducing redundant work.



ROS has three commonly used communication mechanisms: topics, services, and actions. Through the interfaces defined by each communication type, various nodes can be organically connected.

2. Creating Custom Interface Process

The process of creating custom interface messages is similar to writing executable programs in packages. The main steps are:

1. Create an interface package;
2. Create and edit .msg files, .srv files, .action files;
3. Edit configuration files;
4. Build;

5. Test.

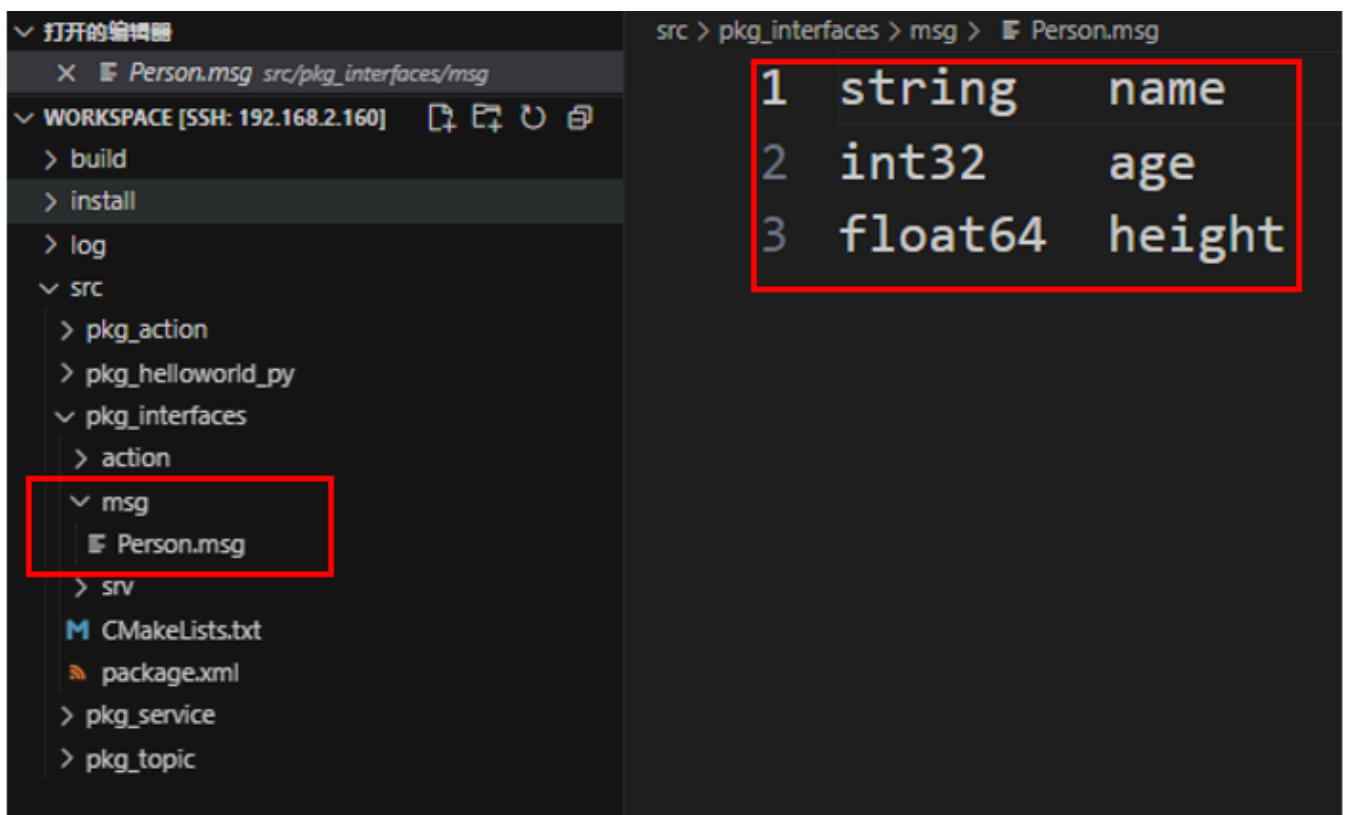
3. Create Action Communication Custom Interface

In the [9. ROS2 Action Communication Server Implementation] example, we have already demonstrated the complete process of creating an action communication interface. You can review it first. We won't repeat it here.

4. Create Topic Communication Custom Interface

1. In the [9. ROS2 Action Communication Server Implementation] course, we have already created the custom interface package. Create a msg folder under the pkg_interfaces package, and create a **Person.msg** file in the msg folder. Enter the following content in the file:

```
string    name
int32     age
float64   height
```



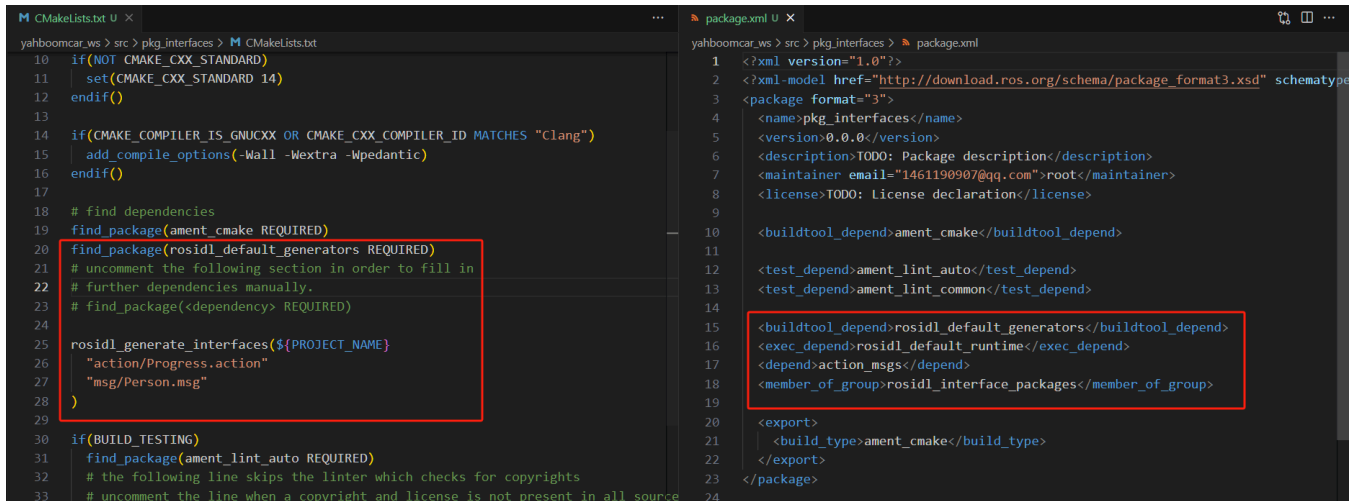
2. Add the following configuration in package.xml and CMakeLists.txt:

- CMakeLists.txt:

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Progress.action"
  "msg/Person.msg"
)
```

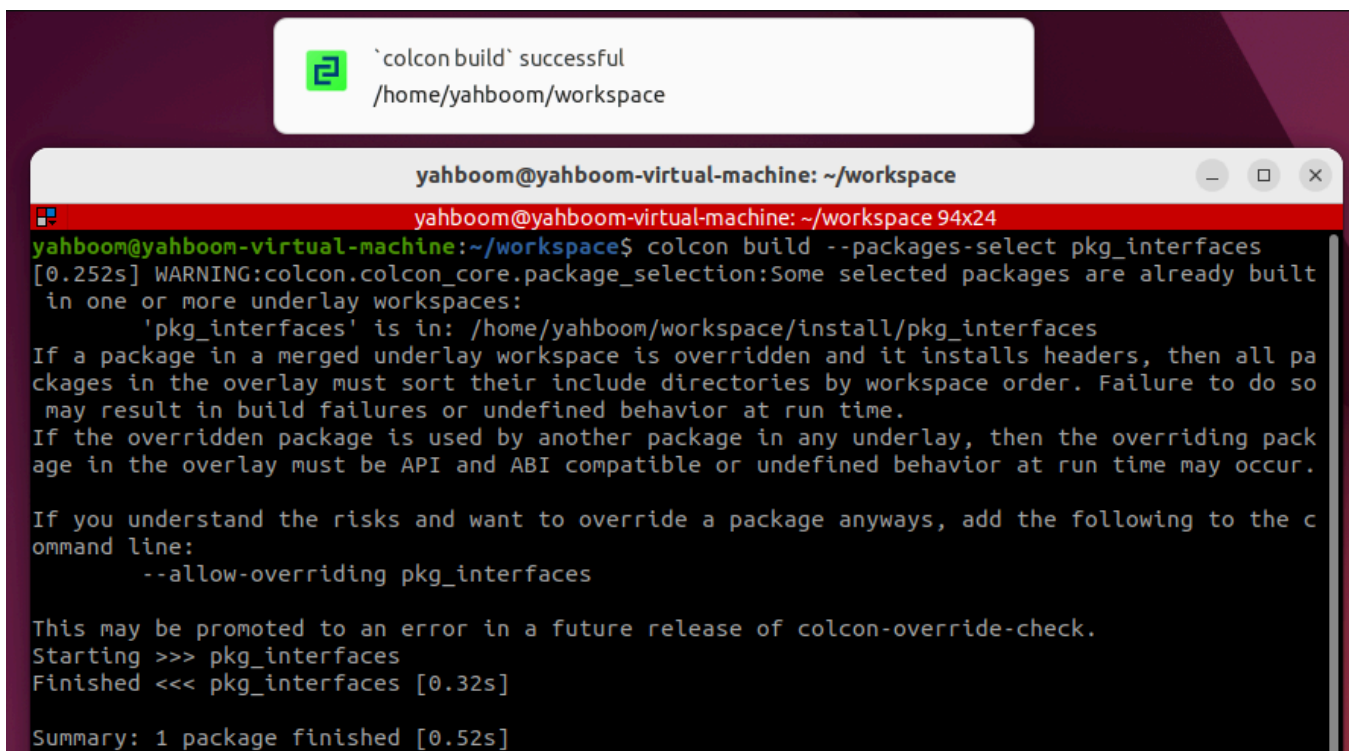
- package.xml:

```
<buildtool_depend>roslint_default_generators</buildtool_depend>
<exec_depend>roslint_default_runtime</exec_depend>
<depend>action_msgs</depend>
<member_of_group>roslint_interface_packages</member_of_group>
```



3. Navigate to the current workspace in the terminal and build the package:

```
colcon build --packages-select pkg_interfaces
```



4. Test if the interface is working correctly

- First, refresh environment variables:

```
source install/setup.bash
```

- View interface type:

```
ros2 interface show pkg_interfaces/msg/Person
```

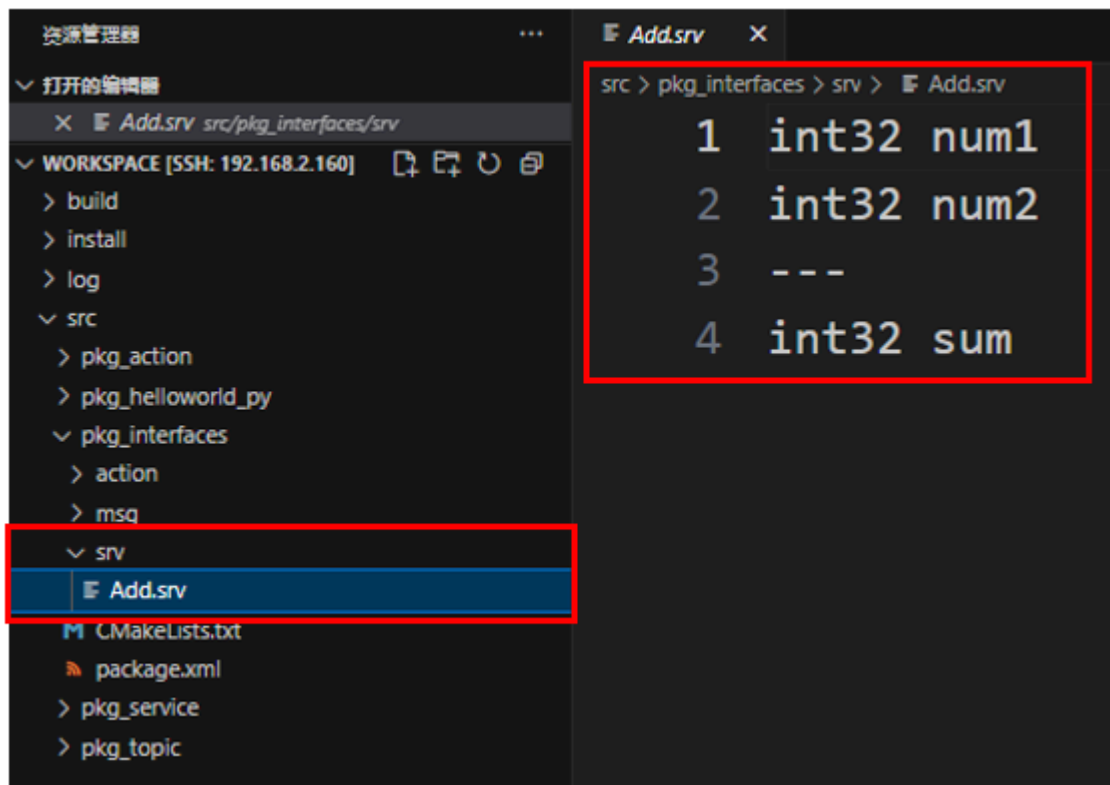
Under normal circumstances, the terminal will output content consistent with the `Person.msg` file.

```
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 interface show pkg_interfaces/msg/Person
string  name
int32   age
float64 height
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws#
```

5. Create Service Communication Custom Interface

1. In the [9. ROS2 Action Communication Server Implementation] course, we have already created the custom interface package. Create an `srv` folder under the `pkg_interfaces` package, and create an `Add.srv` file in the `srv` folder. Enter the following content in the file:

```
int32 num1
int32 num2
---
int32 sum
```



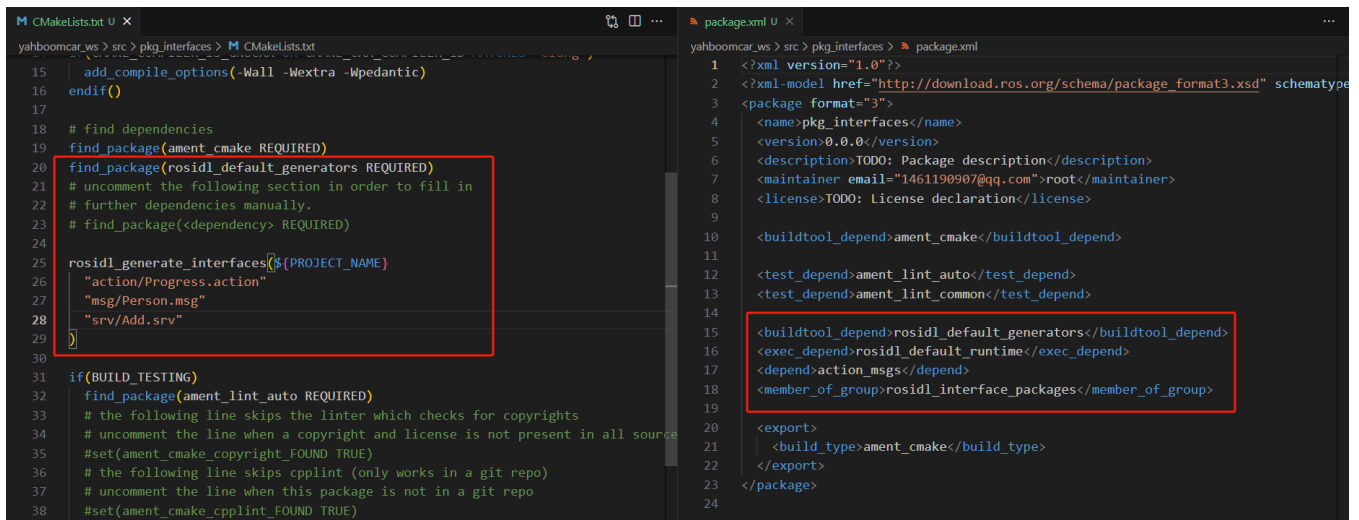
2. Add the following configuration in `package.xml` and `CMakeLists.txt`:

- CMakeLists.txt:

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Progress.action"
  "msg/Person.msg"
  "srv/Add.srv"
)
```

- package.xml:

```
<buildtool_depend>roscpp</buildtool_depend>
<exec_depend>roscpp</exec_depend>
<depend>action_msgs</depend>
<member_of_group>roscpp</member_of_group>
```



3. Navigate to the current workspace in the terminal and build the package:

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws
colcon build --packages-select pkg_interfaces
source install/setup.bash
```

4. Test:

```
ros2 interface show pkg_interfaces/srv/Add
```

Under normal circumstances, the terminal will output content consistent with the `Person.msg` file.

